

UNIT 6:5.1 RABIN KARP ALGORITHM

- Pattern / String matching Algorithm.

Problem: If a text is given, and a pattern is given, to find out whether that pattern is found in that string (text) or not.

Given: Text: $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \\ a & a & a & a & a & b \end{matrix}$
 pattern: $\begin{matrix} 1 & 2 & 3 \\ a & a & b \end{matrix}$

Let us give our own codes to letters:

a: 1, b: 2, c: 3, ... z: 26.

$\therefore$ the pattern ~~text~~ 'aab' is recognised as:

$$(1+1+2) = 4$$

hashcode $\swarrow$ $\searrow$ hashcode
 function $h(p)$

Now, size of text: $n = 6$.

size of pattern: $m = 3$.

Sliding of values is called "rolling hash function".

Step 1) $\begin{matrix} a & a & a & a & a & b \\ \underline{1+1+1} & & & & & \\ & 3 & & & & \end{matrix} \neq 4$

Step 2) $\begin{matrix} a & a & a & a & a & b \\ \underline{1+1+1} & & & & & \\ & 3 & & & & \end{matrix} \neq 4$

Step 3) $\begin{matrix} a & a & a & a & a & b \\ \underline{1+1+1} & & & & & \\ & 3 & & & & \end{matrix} \neq 4$

step 4) a a a a b
 $1+1+2 = 4 = 4.$

∴ hash code is equal
 and pattern is also same

∴ pattern is found in the main text from
 index 4 to 6.

Ex 2: a b c d a b c e
 pattern: b c e
 $2 + 3 + 5 = 10$

step 1: a b c d a b c e
 $1+2+3 = 6 \neq 10.$

step 3) a b c d a b c e
 $3+4+1 = 8 \neq 10.$

step 2: a b c d a b c e
 $2+3+4 = 9 \neq 10.$

step 4) a b c d a b c e
 $4+1+2 = 7 \neq 10$

step 5: ∴ step 6) a b c d a b c e
 $2+3+5 = 10 = 10.$

and pattern is also matching.

∴ pattern found in text from index 6 to 8.

Time / Space Complexity: $O(n-m+1)$

Now, there is a drawback:

Ex: text: c c a e c a a e d b a.

pattern: d b a; $4+2+1=7$

step 1) c c a = $3+3+1=7=7$; but eca $\neq$ d b a.

step 2) c a e = $3+1+3=7=7$; but cae $\neq$ d b a.

So continuously, ~~some~~ hash codes are matching, but is
 always colliding. [Not the same pattern but same
 hash codes].

→ This is called spurious hits.

∴ Here, worst $T(n) = O(mn)$

∴ This simple hash function cannot be taken.

Hash-Function Calculation by Rabin Karp:

Given: text: c c a c c a a e d b a.

pattern: d b a.

$$\begin{aligned}\text{Let hash function of 'dba' be: } & d \times 10^2 + b \times 10^1 + a \times 10^0 \\ & = 4 \times 10^2 + 2 \times 10^1 + 1 \times 10^0 \\ & = 400 + 20 + 1 = 421.\end{aligned}$$

[Note: Here, The base value is '10'. We can change that however we like, supposedly chosen depending upon number of characters we are working with.

$\therefore$ we can also take base case as 255 for all characters]

[For considering everything, even special chars, take base as 127].

Step 1) c c a c c a a e d b a
 $3 \times 10^2 + 3 \times 10^1 + 3 \times 10^0 = 331 \neq 421.$

Step 2) c c a c c a a e d b a
 $3 \times 10^2 + 1 \times 10^1 + 3 \times 10^0 = 313 \neq 421$

$\vdots$

Step Final) c c a c c a a e d b a
 $444 \neq 421.$

So here, Spurious hits are less, and $\therefore$ pattern is found.

Also, we think hash function is too large, we can apply a $\% (\text{mod})$ func. after calculating the hash function.

But, again,

Chances of spurious hits will increase.

Ex: text : 3 1 2 3 4 8 6 2.

pattern : 2 3 4.

$q = 13$ (bits).

$h_p, h_t = p \% q = 234 \% 13 = 0$.

2, $312 \rightarrow 312 \% 13 = 0$ (Spurious hit)

$123 \rightarrow 123 \% 13 = 6$.

$234 \rightarrow 234 \% 13 = 0$ (~~Spurious~~ Valid Hit)

$348 \% 13 = 10$

$486 \% 13 = 5$

$862 \% 13 = 4$.

5.2 VERTEX CORNER:

ROBIN-KARP ALGORITHM:

Robin-karp (T, P)

$n = P.length$

$m = P.length$

$h_p = Hash(P[...])$

$h_t = Hash(T[...])$

For $s = 0$ to $n - m$

if ($h_p == h_t$)

if ($P[0..m-1] = T[s+0..s+m-1]$)

print ("pattern found with shift", s)

if ($s < n - m$)

$h_t = Hash(T[s+1..s+m])$.

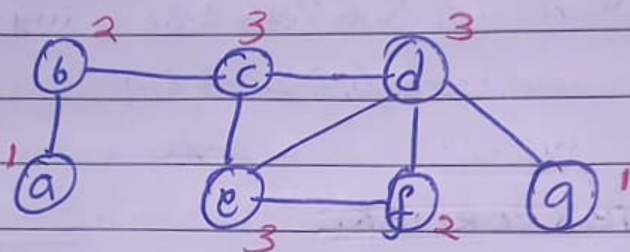
5.2 VERTEX COVER.

The vertex cover is a fundamental problem in graph theory. It is a set of vertices such that each edge of the graph is incident to at least one vertex in the set.

The minimum vertex cover problem seeks to find the smallest possible vertex cover in a given graph.

This is a NP-Hard Problem

Given Graph:



Here,

$$E' = \{ (a,b), (b,c), (c,d), (c,e), (e,f), (d,f), (d,g) \}$$

Step 1) Taking "d"

Step 2) taking "e"

Step 3) taking "b"

$\therefore$ Vertex Cover: $\{d, e, b\}$

Time Complexity: $O(m)$ for a graph, $m = \text{no. of edges}$
 $n = \text{no. of nodes}$

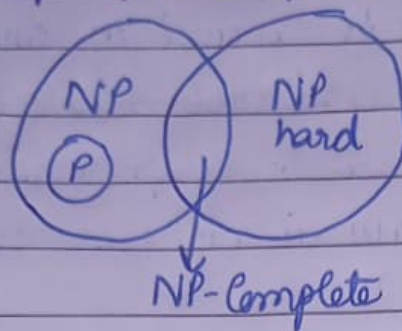
Worst case: $m = O(n^2)$

ALGORITHM:Approx Vertex Cover Problem ($G = (V, E)$)

1. $C \leftarrow \emptyset$
2. $E' \leftarrow E(G)$
3. while ($E' \neq \emptyset$)
4. do let (u, v) an arbitrary edge in E' .
5. $C \leftarrow C \cup \{u, v\}$
6. Remove from E' every edge incident either u and v .
7. Return C .

SATISFIABILITY (SAT) & CNF (Conjunction Normal Form):If a formula evaluates to TRUE $\Rightarrow$ satisfiable.If a formula evaluates to FALSE $\Rightarrow$ unsatisfiable.Ex: $A \Rightarrow \text{True}$; $B \Rightarrow \text{False}$; $\neg \rightarrow$ opposite.① $F = A \cap \bar{B}$; $T \cap T \Rightarrow \text{True} \Rightarrow \text{satisfiable}$.② $F = A \cap \bar{A} \Rightarrow T \cap F \Rightarrow \text{False} \Rightarrow \text{unsatisfiable}$. $\vee \Rightarrow \text{OR}$, $\wedge \Rightarrow \text{And}$.③ If $X_1 \Rightarrow \text{False}$ and $X_2 \Rightarrow \text{True}$.Find: $F(X_1, X_2) \cap (X_2 \vee \bar{X}_1) \cap (\bar{X}_1 \vee \bar{X}_2)$ $= \text{True}$ $= (F \vee T) \cap (T \vee T) \cap (T \vee F)$ $= (T) \cap (T) \cap (T) = T \Rightarrow \text{satisfiable}$.

5.3 Types of Complexity Classes:



→ P class

→ NP class

→ Co NP Class

→ NP-Hard

→ NP-Complete.

(1) P-Class

- ° P stands for polynomial time in the P-Class, which are easily solvable in polynomial time.
- ° problems with a yes or no answer can be solved by a deterministic m/c in polynomial time.

Features:

- (i) Simple to find
- (ii) Can be a class of feasible & solvable Computing Problems
- (iii) It means that the ^{problems} ~~issues~~ can be both theoretically & practically solvable.

Example: Fractional Knapsack.

MST

Sorting

Travelling salesman

(2) NP-Class

- A problem which cannot be solved in polynomial time, but verified in polynomial time.
- Answers can be checked in polynomial time.
- NP stand for non-deterministic polynomial time in the NP-class.
- It is the set of decision problems that can be solved in polynomial time by a non-deterministic m/c.

Features:

- NP Class solutions are challenging to find.
- This is because they are solved by a non-deterministic m/c.
- They are simple to prove.
- A Turing m/c can verify NP problems in polynomial time.

Ex: Graph Colouring.

Boolean Satisfiability Problem (SAT)

A given boolean expression determining if there exists a truth assignments to its variables for which the expression is true is defined as the satisfiability problem.

Ex:	a	b	c	$(a \wedge b) \vee c$	$(a \wedge \neg a) \vee (c \wedge \neg c)$
	0	0	0	0	$0 \vee 0 \rightarrow 0$
	0	0	1	1	$0 \vee 0 \rightarrow 0$
	0	1	0	0	$0 \vee 0 \rightarrow 0$
	0	1	1	1	$0 \vee 0 \rightarrow 0$
	1	0	0	0	$0 \vee 0 \rightarrow 0$
	1	0	1	1	$0 \vee 0 \rightarrow 0$
	1	1	0	1	$0 \vee 0 \rightarrow 0$
	1	1	1	1	$0 \vee 0 \rightarrow 0$

Here, an expression is taken to test whether there exists a truth assignments to variables a, b, c.

→ There are 5 satisfying truth assignments for expression → hence satisfiable.
 $(a \wedge \neg a) \vee (c \wedge \neg c) \rightarrow$ non-satisfiable.

3) NP-Hard:

- An NP-hard problem is atleast as hard as the hardest problem in NP.
- It is a class of problems such that every problem in NP reduces to NP-Hard.

Features:

- Not all NP-hard problems are NP.
- It takes a long time to check them.
- This means if a solution for a NP-hard problem is given, it takes a long time to check whether it is right or not.
- If there is a polynomial time reduction from problem 'L' to problem 'N' for every problem L in NP, then problem 'N' is NP-hard.

Example: CDP (Clique Decision Problem)

4) NP-Complete:

- NP-Complete is a subset of NP.
- If any ~~problem~~ NP Complete problem can be solved in polynomial time, then every other problem in NP can be solved in polynomial time.

* A decision problem 'L' is NP Complete if it follows the below 2 properties:

- L is in NP.
 - Every problem in NP is reducible to L in polynomial time.
- A problem is NP-Complete if it is both NP and NP-hard.

5.4 Randomized Hiring Problem:5.54 Randomized Quick Sort,

- Randomized Quick Sort Algorithm is where the pivot element is chosen as random instead of using a fixed rule. (Best, last, middle).
- This helps avoid worst-case scenarios (like already sorted arrays) and gives good average performance on all inputs.

#ALGORITHM:

Time Complexity
 Best, Avg: $O(n \log n)$
 Worst: $O(n^2)$
 Space: $O(\log n)$

```
import random
```

```
def randomized_quick_sort(arr, low, high):
```

```
    if low < high:
```

```
        # Randomly choose pivot and partition
```

```
        pivot_index = randomized_partition(arr, low, high)
```

```
        # Recursively sort elements before and after partition
```

```
        randomized_quick_sort(arr, low, pivot_index - 1)
```

```
        randomized_quick_sort(arr, pivot_index + 1, high)
```

```
def randomized_partition(arr, low, high):
```

```
    # Choose a random pivot and swap it with the last element
```

```
    pivot_index = random.randint(low, high)
```

```
    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]
```

```
    return partition(arr, low, high)
```

```
def partition(arr, low, high):
```

```
    pivot = arr[high]
```

```
    i = low - 1
```

```
    for j in range(low, high):
```

```
        if arr[j] <= pivot:
```

```
            i += 1
```

```
            arr[i], arr[j] = arr[j], arr[i]
```

```
    arr[i+1], arr[high] = arr[high], arr[i+1]
```

```
    return i+1
```

—X—